Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Program: Software Engineering

Operating Systems

Laboratory Work No. 4

The xv6 Shell from the Outside

Track B: Build Your Own Small OS (xv6)

Elaborated by:
st. gr. FAF-241
Luchiciov Alexei

Verified by:
Reiţman Patricia

Chișinău – 2026

Contents

1 Introduction 2
2 Fixing sleep for the Current xv6 2
2.1 Verification and Test 2
3 bash versus the xv6 Shell 3
3.1 Comparison Table 3
3.2 Observations 4
4 Who Expands *? The args Program 5
4.1 xv6 args Program 5
4.2 Linux args Program 6
4.3 Observations 6
5 Reading the xv6 Shell 7
5.1 Observations 7
6 Conclusion 8

1. Introduction

The purpose of this laboratory work was to compare the simple xv6 shell with Bash and to understand which tasks are performed by the shell before a program starts. During the work I checked the sleep program from Lab 2, compared common commands in both shells, wrote a small program that prints its command-line arguments, and inspected the source code of user/sh.c.

The practical work was performed in Ubuntu and in xv6 running under the QEMU RISC-V emulator. The comparison showed that xv6 supports the basic Unix mechanisms needed for redirection, pipes, command sequences and background execution, while Bash adds more convenient processing such as globbing, variables, quote handling and command lookup through PATH.

2. Fixing sleep for the Current xv6

The first task was to verify that the sleep program created in Lab 2 still matches the current xv6 interface. I checked the system-call declarations with:

```
cd ~/xv6-riscv
grep -n "pause\|sleep" user/user.h

25:int pause(int);
```

Figure 1.1 – The waiting system call declared in user/user.h

The current tree declares pause(int), so the working sleep program uses pause() instead of the older sleep() system call. The final user/sleep.c was:

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }

  pause(atoi(argv[1]));
  exit(0);
}
```

Figure 1.2 – Updated sleep.c using pause()

2.1. Verification and Test

The Makefile already contained the sleep program in UPROGS, so xv6 was started with make qemu. The two required tests produced:

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ sleep 20
$ sleep
usage: sleep <ticks>
```

Figure 1.3 – sleep running with and without an argument

3. bash versus the xv6 Shell

The same commands were executed first in Bash on Ubuntu and then at the xv6 $ prompt. This made it possible to see which features belong to the basic shell design and which conveniences are added by Bash.

3.1. Comparison Table

Command	Feature	bash	xv6
echo hello > f ; cat < f	redirection	hello	hello
ls | grep READ	pipe	README	README (with xv6 file metadata)
(echo a; echo b) | wc	grouping + pipe	2 2 4	2 2 4
echo one ; echo two	sequence	one / two	one / two
echo bg &	background	job id + bg	bg
ls *	globbing	README f	ls: cannot open *
echo $HOME	variables	/home/alex_007m	$HOME
echo "a b"	quotes	a b	"a b"
mkdir d ; cd d ; ls ..	command lookup	README d f	exec ls failed
arrow up, Tab, Ctrl+R	history / completion	Bash feature	not preserved in log; xv6 has no Bash-style support

Figure 2.1 – Results of the same shell operations in Bash and xv6

The commands for redirection, pipes, grouping, sequences and background execution worked in both environments. The differences became clear for globbing, variables and quotes. In Bash, * was expanded to file names, $HOME was replaced with the home-directory path, and the quotes around "a b" were removed before echo received the argument. In xv6, these strings remained literal.

```
bash:
$ ls *
README  f
$ echo $HOME
/home/alex_007m
$ echo "a b"
a b

xv6:
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
```

Figure 2.2 – Globbing, variables and quotes in the two shells

Another important difference appeared after changing to directory d in xv6. Running ls failed, but using the relative path ../ls worked:

```
$ mkdir d
$ cd d
$ ls ..
exec ls failed
$ ../ls ..
.              1 1 1024
..             1 1 1024
README         2 2 2441
...
```

Figure 2.3 – ls failing after cd d and the working relative-path alternative

3.2. Observations

1. Both shells support ordinary program execution, input/output redirection, pipes, command lists with semicolon, grouping with parentheses, background execution with &, and changing directories. Bash additionally performs filename globbing, variable expansion, quote processing, PATH-based command lookup, and provides a richer interactive interface such as history and completion.
2. After cd d, the command ls fails in xv6 because the shell does not search a PATH variable for executables. The xv6 user programs are stored in the root directory, so from inside d the command can be run as ../ls or ../ls ... .
3. The missing feature I would notice most in daily work is PATH-based command lookup, because without it ordinary commands stop working after changing directories. A simple search through a small list of executable directories would be one of the more direct improvements to add.

This comparison showed that xv6 implements the essential mechanisms of a Unix shell, but leaves out many conveniences that Bash performs before executing programs.

4. Who Expands *? The args Program

To see exactly what a program receives in argv, I added a small user program called args to xv6 and a similar program for Linux. The program prints argc and every argument without changing them.

4.1. xv6 args Program

```
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
    printf("argc = %d\n", argc);

    for(int i = 0; i < argc; i++)
        printf("argv[%d] = [%s]\n", i, argv[i]);

    exit(0);
}
```

The program was added to UPROGS as $U/_args\. After rebuilding xv6, ls showed the new args executable in the filesystem. The important test was:

```
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]

$ args $HOME
argc = 2
argv[1] = [$HOME]

$ args 'single quoted'
argc = 3
argv[1] = ['single]
argv[2] = [quoted']
```

Figure 3.1 – Arguments received by the program inside xv6

4.2. Linux args Program

The Linux version used <stdio.h> and return 0. It was compiled with gcc -o args args_linux.c. In the working directory there were three .c files: args_linux.c, test1.c and test2.c. The same important test produced:

```
$ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [test1.c]
argv[3] = [test2.c]
argv[4] = [x y]

$ ./args $HOME
argc = 2
argv[1] = [/home/alex_007m]

$ ./args 'single quoted'
argc = 2
argv[1] = [single quoted]
```

Figure 3.2 – Arguments received by the Linux program in Bash

4.3. Observations

1. For args *.c "x y", the xv6 program received 4 arguments in total: the program name, the literal string *.c, and two separate pieces containing the quote characters. The Bash/Linux program received 5 arguments because *.c expanded to three file names and "x y" became one argument without quotes.
2. The comparison shows that the shell, not the program and not the kernel, performs expansions before starting a program. Bash expands * and $HOME and removes quote syntax. The xv6 shell does not implement those transformations, so it passes the text much more literally.
3. The saved terminal log contains the words one, two and three as separate invocations rather than one command with several spaces. However, the later inspection of sh.c shows that its tokenizer skips repeated whitespace around tokens, so extra spaces act only as separators and are not preserved inside argv.

5. Reading the xv6 Shell

The final part of the laboratory work inspected user/sh.c, the approximately 500-line program that implements the xv6 shell. The file contained 499 lines in my xv6 tree.

```
$ wc -l user/sh.c
499 user/sh.c

$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Figure 4.1 – Command kinds defined by the xv6 shell

The runcmd() switch executes these command structures at the following locations:

```
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101: case PIPE:
125: case BACK:

Token parsing also recognizes:
286: case '|':
287: case '(':
288: case ')':
289: case ';':
290: case '&':
291: case '<':
294: case '>':
```

Figure 4.2 – Where command kinds and special symbols are handled

Searching for cd showed its special handling in the main shell loop around line 170:

```
$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
```

Figure 4.3 – cd is handled directly by the shell

Searching for * produced many matches because * is also the C pointer and dereference operator. None of the displayed code was a filename-globbing implementation. Searching for $ found only the code that prints the shell prompt:

```
$ grep -n '\$' user/sh.c
137:  write(2, "$ ", 2);
```

5.1. Observations

1. The five command kinds are EXEC for an ordinary command, REDIR for redirection (<, > and >>), PIPE for |, LIST for ;, and BACK for &. Parentheses are parsed for grouping, but the executed command tree is still made from these main command kinds.
2. cd is handled inside the shell itself rather than as a normal external program. The current working directory belongs to the shell process; if cd ran only in a child process, the child could change its own directory but the parent shell would remain in the old directory.

3. The source search did not reveal logic for globbing * or variable expansion $. This explains the Part 2 results: xv6 passed * and $HOME literally instead of transforming them like Bash.

6. Conclusion

In this laboratory work I compared Bash with the xv6 shell and saw which responsibilities belong to a shell. xv6 already supports important Unix mechanisms such as redirection, pipes, sequences, grouping and background execution, but its interface is intentionally much simpler than Bash.

The args experiment made the difference especially clear. In Bash, the shell expanded *.c into file names, expanded $HOME, and removed quote syntax before the program started. In xv6, these transformations were not implemented, so the program received the literal text. I also saw why command lookup fails after cd d: xv6 does not search PATH, so an executable must be referenced by a valid path such as ../ls.

Finally, reading user/sh.c connected the observed behavior with the source code. The five main command types, the special handling of cd, and the absence of globbing and variable-expansion code explain the results seen from the outside. This laboratory helped me understand that a shell is not only a program launcher: it parses a command line, builds operations such as pipes and redirections, and may also transform arguments before programs receive them.